

One Function, Three Spellings

Where the types sit, how each line is read, and why one of them will not compile

October 2026

The function

All three lines below define the same thing: an async function that takes a number `x` and resolves to `x * 2`. Its type, in every case, is a function from a number to a promise of a number, written `(x: number) => Promise<number>`.

```
// Form 1: types on the function, variable inferred
const f = async (x: number): Promise<number> => { return x * 2 };

// Form 2: types on the variable, function bare
const f: (x: number) => Promise<number> = async (x) => { return x * 2 };

// Form 3: function declaration (no variable at all)
async function f(x: number): Promise<number> { return x * 2 }
```

Vocabulary used below

Word	Means	In these examples
the arrow / the arrow function	a function written with <code>=></code>	<code>async (x) => { ... }</code>
the async function	a function marked <code>async</code> ; always returns a Promise	the same thing, named by its keyword
the value / the right-hand side	what comes after <code>=</code>	the same thing, named by its position
the variable	the name being declared	<code>f</code>
the variable's type	what comes between <code>:</code> and <code>=</code> on the left	<code>(x: number) => Promise<number></code>
<code>Promise<number></code>	a placeholder that will later hold a number	the return type of any async function returning a number

Form 1: types on the function

```
const f = async (x: number): Promise<number> => { return x * 2 };
      ^      ^^^^^^  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
      2      async parameter return type      => body
              keyword with type
```

Token by token

Piece	Say	Slot
<code>const f</code>	"constant <code>f</code> "	1, 2
<code>=</code>	"is set to"	border
<code>async</code>	"an async function"	4
<code>(x: number)</code>	"taking a number <code>x</code> "	4
<code>: Promise<number></code>	"that returns a promise of a number"	4
<code>=></code>	"whose body is"	4
<code>{ return x * 2 }</code>	"return <code>x</code> times 2"	4

Read aloud, whole: "Constant `f` is set to an async function taking a number `x`, returning a promise of a number, whose body returns `x` times 2."

Where the types are: both inside the function. The `:` after `x` types the parameter. The `:` after `)` types the return. The variable `f` has no annotation (no `:` between `f` and `=`); TypeScript infers `f`'s type from the arrow: `(x: number) => Promise<number>`.

What is optional here: the `: Promise<number>`. Drop it and TS still infers `Promise<number>` because the function is `async` and returns a number. The `x: number` is not optional in strict mode: with nothing on the left to supply it, a bare `(x)` would be an implicit `any`.

Form 2: types on the variable

```
const f: (x: number) => Promise<number> = async (x) => { return x * 2 };
      ^               ^               ^               ^
      2 3: the TYPE of f           = 4: the VALUE    => body
      (a function type)           async arrow, bare x
```

Token by token

Piece	Say	Slot
const f	"constant f "	1, 2
: (x: number) => Promise<number>	"of type: function from a number x to a promise of a number"	3
=	"is set to"	border
async	"an async function"	4
(x)	"taking x " (its type comes from the left)	4
=>	"whose body is"	4
{ return x * 2 }	"return x times 2"	4

Read aloud, whole: “Constant `f`, of type function-from-number-to-promise-of-number, is set to an async function taking `x` whose body returns `x` times 2.”

Where the types are: everything between `:` and `=` is the type of `f`. The arrow on the right has no annotations at all. `x` is still a `number`, because TS reads the expected type on the left and pushes it into the bare parameter. This is called **contextual typing**, and it is the same mechanism that types `x` in `arr.map((x) => x * 2)`: the expected callback type comes from `map`.

Two arrows, two jobs

```
const f: (x: number) => Promise<number> = async (x) => { return x * 2 };
//           ^^ "to"                                ^^ "whose body is"
//           └ TYPE (after the colon) ─┘           └ VALUE (after the equals) ─┘
```

Arrow	Side of =	Job	Say
(x: number) => Promise<number>	left, after :	describes a shape; runs nothing	"number to promise of number"
async (x) => { ... }	right, after =	a real function	"taking x , whose body is ..."

The **=** is the border. Left of it is what **f** IS; right of it is what **f** HOLDS.

Form 3: function declaration

```

async function f(x: number): Promise<number> { return x * 2 }

```

async	keyword	name	param	return type	body

Read aloud: “Async function `f`, taking a number `x`, returning a promise of a number, whose body returns `x` times 2.”

No `const`, no `=`, no arrow. The name `f` is part of the function syntax. The types sit in the same two places as Form 1 (after `x`, after `)`). This is the most common form for a standalone, named function in the code you will review.

The broken one

```
const f: (x: number) => async (x) => { return x * 2 }; // x does not compile
```

Why: there is no `=`. After the `:` TypeScript is reading a **type** and keeps reading until it finds `=`. It sees `(x: number) =>`, the start of a function type, and now expects a *return type*. It gets `async (x) => { ... }`, which is a value, not a type, so it stops with a syntax error. Slot 3 never ended: slot 4 never started.

```
const f: (x: number) => async (x) => { return x * 2 };
2 3 ..... ? no border, so no slot 4
```

The fix

is to insert the border and finish the type:

```
const f: (x: number) => Promise<number> = async (x) => { return x * 2 };
//                                     ^
//                                     finish the type the border
```

Side by side

	Form 1	Form 2	Form 3
Written	<code>const f = async (x: number): Promise<number> => { ... }</code>	<code>const f: (x: number) => Promise<number> = async (x) => { ... }</code>	<code>async function f(x: number): Promise<number> { ... }</code>
Where <code>x: number</code> sits	on the arrow	on <code>f</code> 's type	on the declaration
Where <code>Promise<number></code> sits	after the arrow's <code>)</code>	on <code>f</code> 's type, after <code>=></code>	after the declaration's <code>)</code>
Arrow parameters	typed	bare (contextually typed)	n/a
Has a <code>=</code>	yes	yes	no
Number of <code>=></code>	one (value)	two (one type, one value)	none
Type of <code>f</code>	inferred from the arrow	declared; the arrow must match it	from the declaration
<code>this</code> inside	the surrounding <code>this</code>	the surrounding <code>this</code>	depends on how it is called
Hoisted (usable before the line)	no	no	yes
Meaning	identical	identical	identical

Reading rules

1. **Find the `=`**. Left of it (after `:`) is the type and runs nothing. Right of it is the value. No `=`: either a `function` declaration (Form 3) or a callback passed as an argument, or a syntax error.
2. **Two `=>` on one line** means one is in the type and one is in the value; the `=` is between them. Say “to” for the type arrow and “whose body is” for the value arrow.
3. **A bare `(x)` is not `any`**. Ask where its type comes from: the variable’s annotation on the left (Form 2), or the method it is passed to (`map`, `then`, `app.get`). Only a bare parameter with nothing on either side is an implicit `any`.
4. **A missing `: Promise<number>` is not a bug**. It is inferred. To work it out yourself: find every `return`; if the function is `async`, wrap the result in `Promise<...>`.
5. **`async` means the return type always starts with `Promise<`**, whether written or inferred. Every caller must `await` it; a caller that uses the result directly is holding the promise, not the value.

Where each form shows up in real code

You will see	Form	Note
<code>export async function getUser(id: string): Promise<User></code>	3	most exported functions
<code>const handler = async (req: Request, res: Response) => { ... }</code>	1	handlers assigned to a const
<code>const handler: RequestHandler = async (req, res) => { ... }</code>	2	when a named function type exists; <code>req</code> , <code>res</code> typed by <code>RequestHandler</code>
<code>app.get("/", async (req, res) => { ... })</code>	callback	no variable; types come from <code>app.get</code>
<code>ids.map(async (id) => fetchUser(id))</code>	callback	gives <code>Promise<User>[]</code> ; needs <code>Promise.all</code>
<code>const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms))</code>	1	not async, but returns a Promise explicitly